

TECHNICAL ARTICLE & PRODUCTION RUNBOOK

PostgreSQL Database Bloat

What It Is, How to Detect It, and How to Vacuum Safely

Validated against PostgreSQL 18.6 official documentation

Production baseline: PostgreSQL 18 | September 2026

Original article credit “PostgreSQL Database Bloat: What It Is, How to Detect It, and How to Vacuum Safely” by Rahul Kumar, published July 29, 2026 on LinkedIn. This guide expands, restructures, and validates the subject; the original author retains credit for the source article.

Document Control

Field	Value
Document type	Technical Article and Production Runbook
Original article author	Rahul Kumar
Original publication	LinkedIn — July 29, 2026
PostgreSQL baseline	18.6
Primary authority	PostgreSQL Global Development Group official documentation
Command policy	Inspection scripts are read-only; maintenance commands are clearly marked and require approval

Source and Accuracy Statement

The LinkedIn article supplies the topic and core workflow. PostgreSQL 18 official documentation is the technical authority. Scripts were placed next to the exact inspection, diagnosis, maintenance, or validation step they support. Physical bloat cannot be proven by one statistics counter; the guide separates estimates, physical inspection, root cause, and remediation.

Production rule Start with read-only evidence. Prefer standard VACUUM and healthy autovacuum. Use VACUUM FULL, CLUSTER, or a rewrite only when physical shrinkage is necessary, blocking is approved, and temporary disk/WAL capacity is verified.

Table of Contents / Index

1. Executive Summary	4
2. Bloat, MVCC, and Dead Tuples	4
3. Safety and Evidence-First Workflow	5
4. Baseline Inventory Scripts	6
5. Dead-Tuple and Maintenance Scripts	7
6. Physical Bloat Inspection	8
7. Root-Cause Diagnostic Scripts	9
8. Safe Standard VACUUM	11
9. Autovacuum Tuning	12
10. Index Bloat and REINDEX	13
11. VACUUM FULL and Table Rewrites	14
12. Monitoring During Maintenance	15
13. Troubleshooting Scenarios	16
14. Special Cases and Common Mistakes	17
15. Production Runbook	18
16. Corrections and Validation	19
17. Credits and Official References	20

1. Executive Summary

Bloat is allocated table or index space that is no longer used efficiently. It is related to dead tuples but is not identical to `n_dead_tup`. A relation may contain reusable free space after VACUUM, have low dead-tuple estimates but historical empty pages, or grow because of wide live rows rather than bloat.

Observation	What it proves	What it does not prove
Large relation size	Where storage is allocated.	That the relation is bloated.
High <code>n_dead_tup</code>	Estimated obsolete tuples awaiting/eligible for cleanup.	Exact wasted bytes or file shrink potential.
Standard VACUUM completed	Cleanup/reuse and visibility/freeze work ran.	That the OS file became smaller.
<code>pgstattuple</code> results	A point-in-time physical tuple/free-space view.	Future steady-state size or safe rewrite decision alone.
VACUUM FULL shrank table	Rewrite returned excess space.	That root cause is fixed or size will remain small.

Recommended sequence Inventory size → inspect dead tuples and maintenance history → check physical free space where justified → find blockers and workload cause → run targeted standard VACUUM → tune autovacuum → consider rewrite only for verified permanent shrink need.

2. Bloat, MVCC, and Dead Tuples

MVCC lets transactions see appropriate row versions from their snapshots. UPDATE normally creates a new version; DELETE marks a version obsolete. A version cannot be removed while a relevant snapshot or transaction horizon may still need it. VACUUM later removes dead tuple references, makes heap space reusable, maintains visibility/free-space information, and freezes old transaction metadata.

Type	Typical mechanism	Operational effect
Heap bloat	Dead versions, non-reusable gaps, historical high-water size.	More pages, scan I/O, cache pressure, backup/storage footprint.

Type	Typical mechanism	Operational effect
Index bloat	Dead/empty or sparsely used index pages under churn.	Larger indexes, more I/O/cache use, write maintenance.
TOAST bloat	Churn in out-of-line values for wide attributes.	Hidden storage growth attached to the table.
Catalog bloat	High DDL/temp-object churn and maintenance lag.	Slower catalog access and administrative queries.
Normal free space	VACUUM-reclaimed space intentionally retained for reuse.	Not automatically harmful; may be valuable for future writes.

Important distinction The goal is not zero dead tuples or minimum file size. A healthy system reaches a stable size where routine VACUUM makes space reusable at roughly the rate the workload creates obsolete versions.

3. Safety and Evidence-First Workflow

1. Confirm target database, PostgreSQL version, role, schema, table owner, HA/replication topology, and maintenance window.
2. Capture relation-size history, query latency, storage headroom, backup status, WAL rate, and replica lag.
3. Run only read-only inventory and statistics queries first.
4. Identify long transactions, prepared transactions, replication slots, standby feedback, locks, and autovacuum saturation.
5. Use physical inspection extensions only when approved; understand their scan and privilege cost.
6. Choose the smallest action: let autovacuum work, targeted ANALYZE, targeted standard VACUUM, autovacuum tuning, REINDEX, or planned rewrite.
7. Execute one large relation at a time and monitor progress, I/O, WAL, locks, disk, replicas, and application latency.
8. Validate outcome and root cause; document rollback/recovery and preventive thresholds.

```
SELECT current_database() AS database_name,
       current_user AS connected_user,
       version() AS postgres_version,
       pg_is_in_recovery() AS is_standby;
```

```
SHOW transaction_read_only;
SHOW autovacuum;
SHOW track_counts;
SHOW default_transaction_isolation;
```

Transaction rule VACUUM cannot run inside a transaction block. In psql, verify that an explicit BEGIN is not active. Do not wrap VACUUM commands in a migration tool transaction.

4. Baseline Inventory Scripts

4.1 Largest user tables

```
SELECT s.schemaname,
       s.relname AS table_name,
       pg_size_pretty(pg_table_size(s.relid)) AS table_and_toast_size,
       pg_size_pretty(pg_indexes_size(s.relid)) AS indexes_size,
       pg_size_pretty(pg_total_relation_size(s.relid)) AS total_size,
       pg_total_relation_size(s.relid) AS total_bytes
FROM pg_catalog.pg_statio_user_tables AS s
ORDER BY total_bytes DESC
LIMIT 20;
```

pg_table_size includes the main table, TOAST, free-space map, and visibility map but excludes indexes. pg_total_relation_size adds indexes. Large size is an investigation priority, not proof of bloat.

4.2 Table and index breakdown

```
SELECT c.oid::regclass AS table_name,
       pg_size_pretty(pg_relation_size(c.oid)) AS heap_main,
       pg_size_pretty(pg_table_size(c.oid)) AS table_plus_toast,
       pg_size_pretty(pg_indexes_size(c.oid)) AS all_indexes,
       pg_size_pretty(pg_total_relation_size(c.oid)) AS total
FROM pg_class c
JOIN pg_namespace n ON n.oid=c.relnamespace
WHERE c.relkind IN ('r','m')
      AND n.nspname NOT IN ('pg_catalog','information_schema')
ORDER BY pg_total_relation_size(c.oid) DESC LIMIT 30;
```

4.3 Individual index sizes and use

```
SELECT ui.schemaname, ui.relname AS table_name,
       ui.indexrelname AS index_name,
       pg_size_pretty(pg_relation_size(ui.indexrelid)) AS index_size,
       ui.idx_scan, ui.last_idx_scan, ui.idx_tup_read, ui.idx_tup_fetch
```

```
FROM pg_stat_user_indexes ui
ORDER BY pg_relation_size(ui.indexrelid) DESC LIMIT 50;
```

Usage warning `idx_scan` is cumulative and can reset. Zero does not prove an index is disposable: check constraints, replicas, seasonal jobs, prepared statements, failover workload, and a complete observation period.

5. Dead-Tuple and Maintenance Scripts

5.1 Estimated dead tuples

```
SELECT s.schemaname, s.relname AS table_name,
       s.n_live_tup AS estimated_live_rows,
       s.n_dead_tup AS estimated_dead_rows,
       round(100.0*s.n_dead_tup /
            NULLIF(s.n_live_tup+s.n_dead_tup,0),2) AS dead_row_pct,
       pg_size_pretty(pg_total_relation_size(s.relid)) AS total_size,
       s.last_vacuum, s.last_autovacuum,
       s.last_analyze, s.last_autoanalyze,
       s.vacuum_count, s.autovacuum_count
FROM pg_stat_user_tables s
ORDER BY s.n_dead_tup DESC LIMIT 30;
```

`n_live_tup` and `n_dead_tup` are estimates. They can lag, reset, and differ from physical space. Use them to rank investigation, not to calculate exact bloat or justify `VACUUM FULL`.

5.2 Change rate and HOT updates

```
SELECT schemaname, relname,
       n_tup_ins, n_tup_upd, n_tup_del,
       n_tup_hot_upd, n_tup_newpage_upd,
       n_dead_tup, n_mod_since_analyze, n_ins_since_vacuum
FROM pg_stat_user_tables
ORDER BY (n_tup_upd+n_tup_del) DESC LIMIT 30;
```

5.3 Correctly quoted targeted commands

```
SELECT format('VACUUM (VERBOSE, ANALYZE) %I.%I;',
             schemaname, relname) AS reviewed_command
FROM pg_stat_user_tables
ORDER BY n_dead_tup DESC LIMIT 20;
```

Review the generated text and execute one command at a time. Generation is read-only; the resulting VACUUM command is a maintenance action.

6. Physical Bloat Inspection

6.1 pgstattuple and pgstattuple_approx

The optional pgstattuple extension provides physical tuple and free-space information. CREATE EXTENSION requires appropriate privileges. Exact pgstattuple reads the relation and can be expensive; pgstattuple_approx is faster for heap tables by using the visibility map where possible. Test and schedule large scans.

```
-- One-time, approved extension installation:
CREATE EXTENSION IF NOT EXISTS pgstattuple;

-- Faster approximate heap inspection:
SELECT * FROM pgstattuple_approx('public.orders'::regclass);

-- Exact inspection; can scan the entire relation:
SELECT table_len, tuple_count, tuple_len, tuple_percent,
       dead_tuple_count, dead_tuple_len, dead_tuple_percent,
       free_space, free_percent
FROM pgstattuple('public.orders'::regclass);
```

Metric	Interpretation
table_len	Physical main-fork length inspected.
tuple_percent	Space occupied by live tuples.
dead_tuple_percent	Space occupied by dead tuples at inspection time.
free_percent	Immediately free space inside the heap.
approx_free_percent	Approximate free space from pgstattuple_approx.
dead_tuple_len	Dead tuple bytes—not total reclaimable file bytes.

6.2 B-tree page-level indicators

```
-- B-tree only; inspect one approved index:
SELECT version, tree_level, index_size, root_block_no,
       internal_pages, leaf_pages, empty_pages,
```

```
deleted_pages, avg_leaf_density, leaf_fragmentation
FROM pgstatindex('public.orders_customer_id_idx'::regclass);
```

pgstatindex is B-tree-specific. Low leaf density or fragmentation is context, not an automatic REINDEX order. Fillfactor, build history, workload, deduplication, future reuse, and index type matter. Extension functions require privileges and consume I/O.

6.3 Free-space map sample

```
CREATE EXTENSION IF NOT EXISTS pg_freespacemap;
SELECT blkno, avail
FROM pg_freespace('public.orders'::regclass)
ORDER BY avail DESC LIMIT 50;
```

Measurement rule Physical inspection is a point-in-time sample. Compare with size trends, write pattern, future reuse, scan performance, maintenance history, and the business need to return space to the operating system.

7. Root-Cause Diagnostic Scripts

7.1 Long transactions and old snapshots

```
SELECT pid, username, datname, state, xact_start,
       now()-xact_start AS xact_age,
       age(backend_xid) AS backend_xid_age,
       age(backend_xmin) AS backend_xmin_age,
       wait_event_type, wait_event, left(query,180) AS query
FROM pg_stat_activity
WHERE xact_start IS NOT NULL
      OR backend_xid IS NOT NULL OR backend_xmin IS NOT NULL
ORDER BY xact_start NULLS LAST;
```

7.2 Prepared transactions

```
SELECT gid, prepared, owner, database,
       age(transactionid) AS xid_age
FROM pg_prepared_xacts
ORDER BY prepared;
```

7.3 Replication slots retaining tuples/WAL

```
SELECT slot_name, slot_type, active,
       age(xmin) AS xmin_age,
```

```

age(catalog_xmin) AS catalog_xmin_age,
restart_lsn, confirmed_flush_lsn,
wal_status, safe_wal_size
FROM pg_replication_slots
ORDER BY greatest(age(xmin),age(catalog_xmin)) DESC NULLS LAST;

```

7.4 Per-table autovacuum settings and thresholds

```

SELECT c.oid::regclass AS table_name, c.reltuples, c.relpages,
       c.relallvisible, c.relallfrozen, c.reloptions
FROM pg_class c JOIN pg_namespace n ON n.oid=c.relnamespace
WHERE c.relkind='r' AND n.nspname NOT IN ('pg_catalog','information_schema')
ORDER BY c.relpages DESC LIMIT 50;

SELECT name, setting, unit, source
FROM pg_settings
WHERE name LIKE 'autovacuum%' OR name LIKE 'vacuum_cost%'
ORDER BY name;

```

7.5 Locks and blockers

```

SELECT a.pid, a.username, a.state, a.wait_event_type, a.wait_event,
       pg_blocking_pids(a.pid) AS blocking_pids,
       left(a.query,180) AS query
FROM pg_stat_activity a
WHERE cardinality(pg_blocking_pids(a.pid)) > 0
ORDER BY a.query_start;

```

Root cause	Why cleanup/reuse suffers
Old transaction/snapshot	Dead versions remain potentially visible.
Prepared transaction	Transaction horizon persists until resolved.
Replication slot xmin/catalog_xmin	Retains tuples/catalog tuples; inactive consumer may also retain WAL.
Autovacuum trigger too late	Large table accumulates many changes before threshold.
Workers/I/O saturated	Eligible tables wait or vacuum runs too slowly.
Frequent wide updates	Creates heap/TOAST/index churn and reduces HOT opportunity.
Too many indexes	Every update/delete increases index maintenance and vacuum work.

Root cause	Why cleanup/reuse suffers
Bulk deletion	Creates a one-time high volume of obsolete versions.

8. Safe Standard VACUUM

8.1 Target one relation

```
-- Maintenance action; run outside a transaction block:
VACUUM (VERBOSE, ANALYZE) public.orders;

-- Optional: limit VACUUM/ANALYZE buffer strategy for this command:
VACUUM (VERBOSE, ANALYZE, BUFFER_USAGE_LIMIT '2MB') public.orders;
```

Standard VACUUM permits normal SELECT/INSERT/UPDATE/DELETE, but blocks or conflicts with some DDL and can generate significant I/O. ANALYZE is useful when planner statistics need refresh; it is not required solely to reclaim dead-tuple space.

8.2 Current database

```
-- All eligible relations in the current database only:
VACUUM (VERBOSE, ANALYZE);

-- Command-line alternative; run for each required database:
vacuumdb --dbname=appdb --analyze --verbose --jobs=4
```

VACUUM without a relation list processes eligible tables and materialized views in the connected database that the role can maintain. It does not vacuum every database in the cluster. vacuumdb jobs can parallelize across relations/connections; validate capacity and connection limits.

8.3 Useful options and boundaries

Option	Correct use
PARALLEL n	Parallelizes supported index vacuum/cleanup phases, not heap scan or ANALYZE.
SKIP_LOCKED	Skips a relation that cannot be locked immediately; indexes/children can still block.
INDEX_CLEANUP AUTO	Default decision; forcing OFF can leave index/line-pointer cleanup debt.

Option	Correct use
TRUNCATE false	Avoids tail truncation attempt and its brief ACCESS EXCLUSIVE requirement.
BUFFER_USAGE_LIMIT	Controls the buffer access strategy; too high can evict useful cache.
DISABLE_PAGE_SKIPPING	Only when visibility map corruption is suspected from hardware/software problems.
FREEZE	Aggressive freezing; not a generic bloat fix and redundant with FULL.

9. Autovacuum Tuning

PostgreSQL 18 compares estimated obsolete tuples with a base threshold plus a scale-factor share of reltuples, capped by `autovacuum_vacuum_max_threshold`. It also has insert-driven vacuum thresholds and separate ANALYZE thresholds. Large high-churn tables often need per-table settings because a cluster-wide percentage can represent too many rows.

```
vacuum threshold = least(
  autovacuum_vacuum_max_threshold,
  autovacuum_vacuum_threshold
  + autovacuum_vacuum_scale_factor * pg_class.reltuples
)

analyze threshold = autovacuum_analyze_threshold
  + autovacuum_analyze_scale_factor * pg_class.reltuples

-- Example only: derive values from measured churn and vacuum duration.
ALTER TABLE public.orders SET (
  autovacuum_enabled = true,
  autovacuum_vacuum_scale_factor = 0.02,
  autovacuum_vacuum_threshold = 5000,
  autovacuum_analyze_scale_factor = 0.01,
  autovacuum_analyze_threshold = 5000
);

SELECT relname, reloptions
FROM pg_class WHERE oid='public.orders'::regclass;
```

Area	Validation
Trigger timing	Does vacuum begin before dead tuples/size accelerate?

Area	Validation
Worker capacity	Are all workers busy while eligible tables wait?
I/O throttling	Does maintenance meet cadence without harming foreground latency?
Memory	autovacuum_work_mem can be allocated per worker; avoid multiplication risk.
Insert-only tables	Review insert-triggered vacuum for visibility and freezing.
Logging	Use log_autovacuum_min_duration to capture duration, skips, and outcomes.
Partitioned parents	Autovacuum handles partitions but not ANALYZE of storage-less parent; schedule it.
Temporary tables	Session must VACUUM/ANALYZE them; autovacuum cannot access them.

Tuning discipline Change one evidence-backed table setting at a time. Observe several cycles and compare dead-tuple growth, vacuum duration, worker saturation, foreground latency, WAL, replica lag, and XID age.

10. Index Bloat and REINDEX

Standard VACUUM removes dead index entries where index cleanup is performed, but it does not always compact an index to minimum size. PostgreSQL documents REINDEX for a bloated index containing many empty or nearly empty pages. Rebuild only after physical/workload evidence and dependency review.

```
-- Inventory before any rebuild:
SELECT indexrelid::regclass AS index_name,
       pg_size_pretty(pg_relation_size(indexrelid)) AS size,
       idx_scan, last_idx_scan, idx_tup_read, idx_tup_fetch
FROM pg_stat_user_indexes
WHERE relid='public.orders'::regclass
ORDER BY pg_relation_size(indexrelid) DESC;

-- Lower write blocking, but more scans/waits/work:
REINDEX (VERBOSE) INDEX CONCURRENTLY public.orders_customer_id_idx;
```

Method	Lock/work profile	Use
REINDEX INDEX	Blocks writes to the table while rebuilding; reads continue.	Maintenance window, fastest conventional rebuild.
REINDEX INDEX CONCURRENTLY	Two scans and transaction waits; reduced write blocking; cannot run in transaction block.	Busy production table with monitored capacity.
REINDEX TABLE	Rebuilds all table indexes; wider impact.	Only when all indexes have justified need.
VACUUM FULL	Rewrites table and rebuilds indexes under ACCESS EXCLUSIVE.	Table shrink plus index rebuild in approved outage.

Concurrent caveats Only one concurrent index build can run on a table at a time. Concurrent rebuild can wait for old transactions and may leave temporary/invalid artifacts after failure. Monitor `pg_stat_progress_create_index` and investigate before retrying.

11. VACUUM FULL and Table Rewrites

VACUUM FULL rewrites a table into a new disk file, returns unused space to the operating system, rebuilds its indexes, and requires ACCESS EXCLUSIVE for the duration. It needs additional disk space for the new table and indexes until the old copies can be released, and it can generate substantial WAL and replica lag.

```
-- HIGH-RISK MAINTENANCE ACTION; approved outage only:
VACUUM (FULL, VERBOSE, ANALYZE) public.orders;

-- Syntactically valid but normally too broad for production:
VACUUM (FULL, VERBOSE, ANALYZE);
```

Decision question	Proceed only when
Must the OS receive space now?	Storage pressure/business requirement is real; reuse is insufficient.
Is deletion permanent?	Table is not expected to regrow to the previous steady-state size.

Decision question	Proceed only when
Can table be unavailable?	ACCESS EXCLUSIVE outage is approved and tested.
Is disk/WAL capacity sufficient?	New table plus indexes, WAL, temp/backup/replica effects are budgeted.
Is recovery verified?	Recent backup and restore/PITR procedure are tested.
Is duration known?	Production-sized restore test provides a credible estimate.
Is root cause fixed?	Autovacuum/workload/horizon issue is addressed to prevent recurrence.

Alternatives Official documentation also identifies CLUSTER and table-rewriting ALTER TABLE variants as ways to rewrite a table. They likewise require ACCESS EXCLUSIVE and extra space. If an entire table is periodically emptied, consider TRUNCATE instead of DELETE followed by vacuum, subject to transaction, lock, foreign-key, identity, and trigger semantics.

12. Monitoring During Maintenance

12.1 Standard VACUUM progress

```
SELECT pid, datname, relid::regclass AS relation, phase,
       heap_blks_total, heap_blks_scanned, heap_blks_vacuumed,
       index_vacuum_count, dead_tuple_bytes, max_dead_tuple_bytes
FROM pg_stat_progress_vacuum
ORDER BY pid;
```

12.2 VACUUM FULL progress

```
SELECT pid, datname, relid::regclass AS relation,
       command, phase, heap_tuples_scanned,
       heap_tuples_written, heap_blks_total, heap_blks_scanned,
       index_rebuild_count
FROM pg_stat_progress_cluster
ORDER BY pid;
```

12.3 REINDEX progress

```
SELECT pid, relid::regclass AS table_name,
       index_relid::regclass AS index_name,
       command, phase, lockers_total, lockers_done,
       blocks_total, blocks_done, tuples_total, tuples_done
FROM pg_stat_progress_create_index
ORDER BY pid;
```

12.4 Database size, WAL, and replication

```
SELECT pg_size_pretty(pg_database_size(current_database())) AS db_size,
       pg_current_wal_lsn() AS current_lsn;

SELECT application_name, state, sync_state,
       sent_lsn, write_lsn, flush_lsn, replay_lsn,
       pg_size_pretty(pg_wal_lsn_diff(sent_lsn, replay_lsn)) AS replay_lag_bytes
FROM pg_stat_replication;

SELECT * FROM pg_stat_wal;
```

Also monitor filesystem headroom, device latency/throughput, CPU, memory, application latency/error rate, connection pressure, locks, archive backlog, backup overlap, and managed-service limits. Database views do not replace operating-system and application telemetry.

13. Troubleshooting Scenarios

Symptom	Likely evidence	Corrective direction
VACUUM completes but file stays large	Dead tuples fall; free space rises; relation size stable.	Normal behavior; allow reuse. Rewrite only for proven shrink need.
Dead tuples remain high	Old xmin, slot, prepared xact, lock, worker/I/O saturation.	Resolve confirmed horizon/capacity issue; rerun targeted VACUUM.
Dead tuples return quickly	High update/delete churn; thresholds too late.	Tune per table; improve batching/HOT/fillfactor/index design.
Autovacuum never appears	Disabled setting/reloption, track_counts off, thresholds unmet, temp table.	Correct configuration and table type expectations.

Symptom	Likely evidence	Corrective direction
VACUUM is slow	Large heap/indexes, repeated index cycles, I/O throttling, locks.	Monitor phase; tune capacity/memory/cost carefully; schedule.
VACUUM blocks DDL	ShareUpdateExclusive conflict with schema change.	Separate maintenance and deployment windows.
Tail truncation causes lock wait	VACUUM attempts end-page truncation.	Use TRUNCATE false where availability outweighs tail reclaim.
Index remains large	Reusable/empty pages or uncommon access pattern.	Measure B-tree; consider targeted REINDEX.
VACUUM FULL waits	ACCESS EXCLUSIVE blocked by active sessions.	Drain application in approved window; inspect blockers.
Disk fills during FULL	New copy/indexes/WAL exceed headroom.	Stop only by approved incident plan; restore capacity/recovery path.
Replica lag grows	Maintenance WAL and replay/storage capacity.	Monitor and pause workload/action under runbook thresholds.
Planner still poor	ANALYZE absent/stale or correlations not modeled.	ANALYZE; statistics target/extended statistics as appropriate.

14. Special Cases and Common Mistakes

Mistake or special case	Correct PostgreSQL 18 view
n_dead_tup equals bloat bytes.	It is an estimated tuple count, not a physical byte measurement.
Large table means bloated table.	Large may be valid live data, TOAST, indexes, or retained reusable space.
Plain VACUUM never returns any OS space.	Usually it retains space, but it can truncate contiguous empty tail pages when enabled/lock succeeds.

Mistake or special case	Correct PostgreSQL 18 view
VACUUM FULL is the normal cure.	Routine design should use standard VACUUM/autovacuum; FULL is exceptional.
ANALYZE reclaims storage.	ANALYZE updates planner statistics; VACUUM performs cleanup.
PARALLEL vacuum scans heap in parallel.	Parallel workers apply to supported index vacuum/cleanup phases.
Stop PostgreSQL before VACUUM.	VACUUM requires the server running. Drain application only for planned exclusive work.
Terminate long transactions automatically.	Confirm owner/business impact; termination rolls back active work.
Drop inactive slot automatically.	It may require replica/subscriber rebuild and can affect recovery; verify ownership.
Partitioned parent is autovacuumed/analyzed normally.	Partitions are processed; parent has no tuples and needs manual ANALYZE when distribution changes.
Temporary tables are handled by autovacuum.	Autovacuum cannot access other sessions' temporary tables.
Database-wide VACUUM handles every database.	It handles only the connected database; use vacuumdb/loop with controls for the cluster.
VACUUM FULL is appropriate for wraparound emergency.	Official guidance warns against FULL in that emergency; targeted/database-wide standard VACUUM is the recovery path.

15. Production Runbook

15.1 Pre-change

- ☐ Change/incident ticket, table owner, application owner, DBA, and rollback owner assigned.
- ☐ Exact version/database/schema/relation and maintenance privilege verified.
- ☐ Recent backup and restore/PITR validation confirmed for rewrite operations.

- ☐ Size trend, dead tuples, physical inspection, workload cause, and business objective documented.
- ☐ Long transactions, prepared xacts, slots, locks, autovacuum capacity, and replicas reviewed.
- ☐ Disk, WAL, archive, temp, backup, and replica headroom verified.
- ☐ Production-sized test records duration, lock, WAL, and post-operation size.
- ☐ Monitoring dashboards and abort/escalation thresholds ready.

15.2 Execution

- ☐ Start with one schema-qualified relation.
- ☐ Confirm no explicit transaction block.
- ☐ Run targeted standard VACUUM unless approved evidence requires another action.
- ☐ Monitor progress phase, locks, I/O, WAL, disk, replica lag, and application latency.
- ☐ Do not run concurrent deployments, bulk jobs, backups, or competing maintenance unless planned.
- ☐ Communicate milestones and deviations; follow approved stop/escalation rules.

15.3 Post-change validation

```
SELECT schemaname, relname, n_live_tup, n_dead_tup,
       last_vacuum, last_autovacuum, last_analyze, last_autoanalyze
FROM pg_stat_user_tables
WHERE schemaname='public' AND relname='orders';

SELECT pg_size_pretty(pg_table_size('public.orders')) AS table_size,
       pg_size_pretty(pg_indexes_size('public.orders')) AS index_size,
       pg_size_pretty(pg_total_relation_size('public.orders')) AS total_size;
```

- ☐ Target query plans/latency and application health meet baseline or objective.
- ☐ Dead-tuple trend and free-space/reuse behavior are understood.
- ☐ Autovacuum settings and subsequent cycle are validated.
- ☐ Replication/archive/backup status and disk headroom returned to normal.
- ☐ Change record includes commands, start/end, outcome, metrics, issue, rollback, and prevention.

16. Corrections and Validation

Source concept	Validated result / clarification
Correct term is database bloat.	Correct; “blotting” is not PostgreSQL terminology.

Source concept	Validated result / clarification
MVCC leaves obsolete row versions.	Correct; removal waits until versions are no longer visible to relevant snapshots.
n_dead_tup identifies bloat.	Useful prioritization estimate, but not physical bloat measurement.
Plain VACUUM does not shrink OS files.	Generally correct; tail truncation can return contiguous empty end pages when enabled and lock succeeds.
VACUUM ANALYZE refreshes planner statistics.	Correct; ANALYZE is separate from the space-cleanup purpose.
VACUUM FULL returns space to OS.	Correct; it rewrites and takes ACCESS EXCLUSIVE with extra disk requirements.
Database-wide VACUUM affects current database.	Correct.
Regular VACUUM is online.	Normal DML can continue, but I/O/locks/latency still matter.
Replication slots can block cleanup.	Correct when xmin/catalog_xmin retains horizons; slots can also retain WAL.
Lagging replicas always block cleanup.	Only specific mechanisms such as hot_standby_feedback/slot horizons affect primary cleanup; ordinary replay lag alone is not the same.
Indexes accumulate unused pages.	Possible; inspect by access method/workload. Standard VACUUM cleanup and REINDEX have different purposes.
Stop application for database-wide FULL.	Operationally sensible for planned outage; PostgreSQL itself must remain running.

17. Credits and Official References

17.1 Credits

Item	Credit
------	--------

Item	Credit
Original article	Rahul Kumar — “PostgreSQL Database Bloat: What It Is, How to Detect It, and How to Vacuum Safely,” July 29, 2026.
Original article URL	https://www.linkedin.com/pulse/postgresql-database-bloat-what-how-detect-vacuum-safely-rahul-kumar-8tlyc/
Original author profile	https://www.linkedin.com/in/mlenthusiast/
Technical adaptation	Prepared for Praveen Madupu and validated against PostgreSQL 18.6 official documentation.

17.2 PostgreSQL official references

- **Routine Vacuuming:** <https://www.postgresql.org/docs/18/routine-vacuuming.html>
- **VACUUM:** <https://www.postgresql.org/docs/18/sql-vacuum.html>
- **Vacuum Configuration:** <https://www.postgresql.org/docs/18/runtime-config-vacuum.html>
- **Resource Consumption:** <https://www.postgresql.org/docs/18/runtime-config-resource.html>
- **Monitoring Statistics:** <https://www.postgresql.org/docs/18/monitoring-stats.html>
- **Progress Reporting:** <https://www.postgresql.org/docs/18/progress-reporting.html>
- **pgstattuple:** <https://www.postgresql.org/docs/18/pgstattuple.html>
- **pg_freespacemap:** <https://www.postgresql.org/docs/18/pgfreespacemap.html>
- **pg_visibility:** <https://www.postgresql.org/docs/18/pgvisibility.html>
- **REINDEX:** <https://www.postgresql.org/docs/18/sql-reindex.html>
- **CLUSTER:** <https://www.postgresql.org/docs/18/sql-cluster.html>
- **TRUNCATE:** <https://www.postgresql.org/docs/18/sql-truncate.html>
- **Database Object Size Functions:** <https://www.postgresql.org/docs/18/functions-admin.html#FUNCTIONS-ADMIN-DBSIZE>
- **Statistics Collector Views:** <https://www.postgresql.org/docs/18/monitoring-stats.html#MONITORING-PG-STAT-ALL-TABLES-VIEW>
- **vacuumdb:** <https://www.postgresql.org/docs/18/app-vacuumdb.html>
- **Transaction ID Wraparound:** <https://www.postgresql.org/docs/18/routine-vacuuming.html#VACUUM-FOR-WRAPAROUND>